Scelte Tecniche — FAQ Questionnaire App

FAQ Questionnaire (DeepTrace) · 1.0 — Marzo 2026

1. Linguaggi e Framework

Backend — Java 17 + Spring Boot 3.3

Il backend è sviluppato in **Java 17** con **Spring Boot 3.3.5**, scelto per la sua maturità, l'ecosistema ricco e la facilità di integrazione con i servizi AWS.

Libreria	Versione	Ruolo
Spring Boot Web	3.3.5	REST API
Spring Data JPA / Hibernate	3.3.5	ORM e accesso al database
Spring Validation	3.3.5	Validazione input
Spring Mail	3.3.5	Invio email via SMTP
AWS SDK SES	2.25.53	Invio email via Amazon SES
Flyway	incluso	Migrazioni schema database
iText 7	7.2.5	Generazione report PDF
PostgreSQL Driver	runtime	Connettore JDBC
JUnit 5 (Spring Test)	incluso	Test unitari

Il backend espone una REST API JSON e gestisce: - ricezione e validazione del questionario - calcolo del punteggio (somma delle risposte) - persistenza su PostgreSQL - generazione del report PDF in memoria - invio del report via email - salvataggio locale opzionale del PDF tramite flag runtime

Frontend — Vue 3 + Vite + Tailwind CSS

Il frontend è una **Single Page Application** costruita con **Vue 3** (Composition API), compilata con **Vite** e stilizzata con **Tailwind CSS**.

La scelta di Vue 3 è motivata dalla semplicità del modello reattivo, dalla leggerezza del bundle e dall'ottima integrazione con Vite. Tailwind CSS permette di costruire un'interfaccia responsive senza scrivere CSS personalizzato.

Componenti principali: - Form questionario con validazione in tempo reale e preview punteggio - Tabella storico sottomissioni con paginazione client-side (5 elementi/pagina) - Reinvio PDF dalle sottomissioni - Popup admin per la verifica indirizzi email su AWS SES (apertura contestuale su

errori “email non verificata”)

Mobile — React Native (Expo)

L’app mobile è sviluppata con **React Native** usando il framework **Expo**, che semplifica la compilazione e il testing su dispositivi fisici e simulatori iOS/Android senza necessità di configurare Xcode o Android Studio manualmente.

Librerie principali: - `@react-navigation` — navigazione a tab (bottom tab navigator) - `axios` — chiamate HTTP al backend - `react-native-safe-area-context` — gestione aree sicure su diversi dispositivi

La versione mobile mantiene 3 tab (`Questionario` , `Storico` , `Admin`), con logica funzionale allineata alla web app: - warning visivo quando il questionario è salvato ma l’invio email fallisce - reinvio PDF dallo storico - gestione verifica SES e reinvio di submission pending dalla tab Admin

---

2. Servizi Cloud (AWS)

Amazon RDS — PostgreSQL

Il database in produzione è ospitato su **Amazon RDS** con engine PostgreSQL. La scelta ricade su un’istanza `db.t3.micro` (free tier) nella stessa regione del backend, garantendo bassa latenza e nessun costo di traffico intra-VPC.

La configurazione del provider è selezionabile tramite variabile d’ambiente (`APP_DB_PROVIDER`): in locale si usa un container PostgreSQL, in produzione si punta all’endpoint RDS.

Amazon SES — Simple Email Service

L’invio email avviene tramite **Amazon SES** usando l’AWS SDK v2 (chiamate API dirette, non SMTP). La scelta SES garantisce deliverability elevata, costi molto bassi (0,10 USD ogni 1.000 email) e integrazione nativa con l’ecosistema AWS.

Come fallback, il sistema supporta anche SMTP generico (configurato tramite Spring Mail), selezionabile con la variabile `APP_MAIL_PROVIDER` .

AWS Elastic Beanstalk — Backend

Il backend è deployato come container Docker su **AWS Elastic Beanstalk** (single instance, `t3.micro` free tier). Beanstalk gestisce il provisioning dell’istanza EC2, il load balancer e gli aggiornamenti dell’applicazione tramite CLI (`eb deploy`).

Amazon S3 + CloudFront — Frontend

Il frontend compilato (bundle Vite) è pubblicato su **Amazon S3** come sito statico. **CloudFront** funge da CDN per la distribuzione globale, HTTPS automatico e invalidazione della cache ad ogni deploy.

3. Struttura Generale dell'Applicazione

Architettura a layer

```
[Browser / App Mobile]
        │  HTTP/JSON
        ▼
[Vue SPA  |  React Native]
        │  REST API
        ▼
[Spring Boot REST Controller]
        │
   ┌────┴────┐
   │ Service │  ← logica business, calcolo score, generazione PDF
   └────┬────┘
        │
   ┌────┴────┐
   │   JPA   │  ← repository, entità JPA
   └────┬────┘
        │
   [PostgreSQL]   (locale o AWS RDS)
```

Organizzazione del codice (backend)

```
backend/src/main/java/com/deeptrace/faq/
├── config/          configurazioni Spring (DataSource, CORS, Mail)
├── controller/      REST controller (submissions, admin)
├── dto/             oggetti request/response
├── model/           entità JPA
├── repository/      Spring Data JPA repository
└── service/         logica di business (score, PDF, email)
```

Multi-modulo Maven

Il progetto è strutturato come **Maven multi-modulo** dalla root: - `backend` — modulo Spring Boot - `frontend` — modulo wrapper che esegue la build npm tramite il profilo `frontend-build`

Il modulo mobile (`mobile/`) è un progetto React Native indipendente gestito con npm/Expo, non incluso nel build Maven.

Configurazione per ambiente

Tutte le scelte infrastrutturali (provider DB, provider email) sono selezionabili a runtime tramite variabili d'ambiente nel file `.env`, senza modificare il codice:

Variabile	Valori	Effetto
`APP_DB_PROVIDER`	`local` / `rds`	Seleziona il DataSource da usare
`APP_MAIL_ENABLED`	`true` / `false`	Abilita/disabilita invio email
`APP_MAIL_PROVIDER`	`smtp` / `ses`	Seleziona il provider email
`APP_PDF_SAVE_LOCAL`	`true` / `false`	Abilita salvataggio locale dei report PDF
`APP_PDF_OUTPUT_DIR`	path filesystem	Directory dove salvare i PDF quando il flag e attivo

Endpoint admin SES principali: - `POST /api/admin/ses-verify-email` - `GET /api/admin/ses-verification-status?email=...` - `GET /api/admin/mail-config`

Con `APP_MAIL_ENABLED=false` il report viene comunque creato; se `APP_PDF_SAVE_LOCAL=true` il file viene persistito localmente per debugging o test end-to-end senza invio email.

Migrazioni database

Lo schema è gestito con **Flyway**: ogni modifica al database è versionata in `backend/src/main/resources/db/migration/` e applicata automaticamente all'avvio dell'applicazione, garantendo consistenza tra locale e produzione.

Deploy locale

Per lo sviluppo locale è fornito un `docker-compose.yml` che avvia: - container **PostgreSQL** (con volume persistente) - container **backend** (immagine costruita dal Dockerfile `backend/Dockerfile.local`) - container **frontend** (immagine costruita dal Dockerfile `frontend/`)

I servizi comunicano sulla rete Docker creata automaticamente da Compose.

Strategia di test backend

Il backend include test unitari su tre aree critiche: - **ScoreServiceTest**: validazione input e calcolo punteggio totale. - **EmailServiceTest**: selezione provider (`smtp` / `ses`) e comportamento con invio disabilitato. - **PdfReportServiceTest**: verifica PDF valido (header `%PDF`, contenuti attesi) e salvataggio locale condizionale.